Car Owner Hub — Six-Agent Architecture (DB-Aligned)

Oct 24, 2026 · @Aashish Dhital

Overview and what changed

The six agents stay, but every tool now reads and writes only the seven Django models: `users`, `new_cars`, `used_cars`, `buy_issues`, `sell_issues`, `repair_issues` and `posts`. Each user intent becomes an issue record, and everything other users can see is a published `post` with an approved `post_data` snapshot.

Area	Original architecture	DB-aligned architecture
Marketplace inventory	`vehicle_listings` table	Published `posts` with `issue_type = sell`; `used_cars` stay private
New vehicles	Not modelled	`new_cars` catalog (admin-owned, MSRP is a reference price, not stock)
Buyer discovery by sellers	`buyer_preferences` / `buy_requests`	Published `posts` with `issue_type = buy`; `buy_issues` stay private
Seller / buyer contact	`users` contact preferences	Fields the owner approved into `post_data.contact` at publication
Valuation evidence	`vehicle_comparables`, sold prices	Asking prices in published sell posts + `new_cars.msrp`; always labelled as asking prices
Repair cost	`repair_prices` table	Owner-reported `cost` in published repair posts, labelled anecdotal
Mechanics	`mechanics`, `mechanic_reviews`	External places provider + mechanics named in published repair posts
Conversation memory	`conversations`, `conversation_summaries`	ADK session state + structured fields on the issue (`symptoms`, `ai_analysis`, `resolution_details`)

Area	Original architecture	DB-aligned architecture
Post types	`repair`, `buy`, `sell`, `discussion`	`repair`, `buy`, `sell` only (matches `posts.issue_type`)
Post lifecycle	draft → published	`draft` → `approved` → `published` → `archived`
Repair prerequisite	None	The car must be registered in `used_cars` first (`repair_issues.used_car_id` is required)

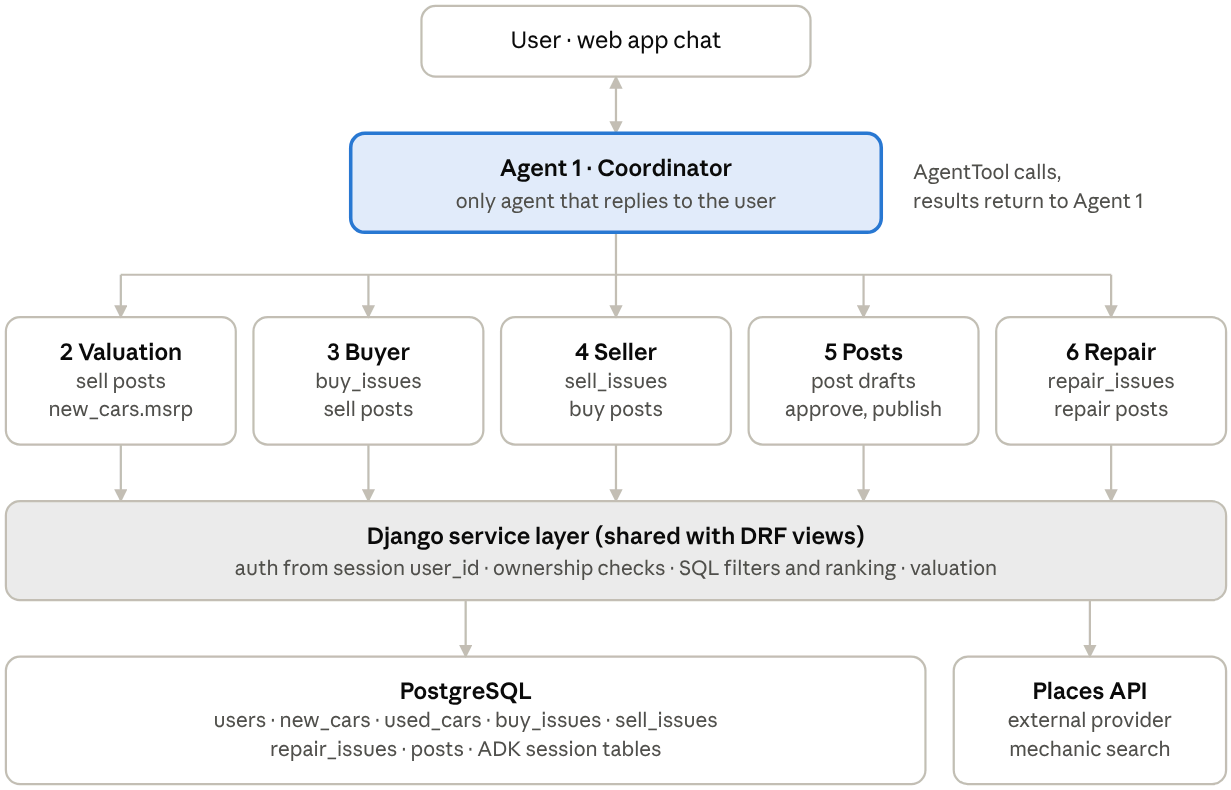
ADK orchestration model

Wrap Agents 2–6 as `AgentTool`s on Agent 1 rather than as transfer sub-agents, so every specialist result returns to the coordinator and only Agent 1 ever speaks to the user. A transfer (`transfer_to_agent`) hands the conversation to the sub-agent, which would break the single-voice rule.

- **Coordinator:** `LlmAgent(name="coordinator", tools=[get_user_context, get_vehicle_details, AgentTool(valuation), AgentTool(buyer), AgentTool(seller), AgentTool(post_gen), AgentTool(repair)])`.
- **Cross-agent sequencing:** Agent 1 chains specialists itself (for selling: Agent 2 → Agent 4 → Agent 5) and passes results through session state, e.g. `state["valuation"]`. Specialists never call each other.
- **Shared deterministic service:** the valuation math lives in one Django service (`valuation_service.estimate()`). Agent 2 owns the explanation; Agent 4 reads the stored result from state, so pricing is never computed twice.
- **Session state (no conversations table):** use ADK's `DatabaseSessionService` on the same PostgreSQL. Its tables are framework-managed and sit outside the seven application models. Keep structured keys: `user_id` (server-set), `active_used_car_id`, `active_issue {type, id}`, `draft_post_id`, `valuation`, `last_search_results`.
- **Durable facts go to the issue, not the chat:** extracted symptoms → `repair_issues.symptoms`; analysis → `ai_analysis`; confirmed fix → `resolution_details`; condition → `sell_issues.condition_details`; budget and preferences → `buy_issues`.

- **Identity:** the DRF layer authenticates the JWT, then creates the ADK session with `state["user_id"]`. Tools read `tool_context.state["user_id"]`; no tool accepts `user_id`, `owner_id` or `buyer_id` as a model argument.
- **Tool execution:** ADK function tools call the Django service layer in-process (the same services DRF views call), so permission and validation rules are defined once.

Only Agent 1 talks to the user; specialists reach the DB through Django



agent topology · 6 agents, 1 service layer, 1 database

Specialists never reach the user or each other; every read and write passes through the same Django services the REST API uses.

Agent 1 — Coordinator (Intent & Recommendation)

Agent 1 resolves which car and which issue the user means, recommends the best next step, delegates, and is the only agent that replies to the user.

Tool	Reads / writes	Purpose
`get_user_context()`	Reads `users`, own `used_cars`, own open `buy_issues` / `sell_issues` / `repair_issues`, own draft `posts`	Resolve "my Civic" to a `used_cars.id`; resume an open issue instead of creating a duplicate
`get_vehicle_details(used_car_id)`	Reads one owned `used_cars` row	Make, model, year, trim, mileage, condition
`register_used_car(make, model, year, mileage, trim?, condition?)`	Writes `used_cars` (owner = session user)	Required before any Sell or Repair issue; confirm details with the user first
`AgentTool(...)` × 5	—	Delegate to Agents 2–6

Intent → issue mapping

User intent	Issue created	Specialist(s)
Buy a car	`buy_issues`	Agent 3
Sell a car	`sell_issues` (needs `used_car_id`)	Agent 2 → Agent 4 → Agent 5
Price check only	None	Agent 2
Car problem	`repair_issues` (needs `used_car_id`)	Agent 6
Share an outcome or request	Post on an existing issue	Agent 5

Workflow: message → `get_user_context()` → resolve or register the car → confirm the intent → delegate → compare specialist results → recommend one option with its evidence and limits, or ask a clarifying question.

Example: "My car won't start; should I repair or sell it?" → Agent 6 creates the repair issue and lists likely causes and cost evidence; Agent 2 values the car; Agent 1 sets repair cost against value and recommends one path.

Agent 2 — Vehicle Valuation

Agent 2 returns an indicative asking-price range built only from published sell posts and the `new_cars` reference MSRP; with the current schema it cannot claim sold-price evidence.

Tool	Reads / writes	Purpose
`get_vehicle_details(used_car_id)`	Reads owned `used_cars`	Subject vehicle attributes
`search_comparable_posts(make, model, year_range, mileage_range, location?)`	Reads `posts` where `issue_type = 'sell'` and `status = 'published'`, filtering on `post_data` (JSONB)	Comparable asking prices with post IDs
`get_catalog_reference(make, model, year, trim?)`	Reads `new_cars.msrp`	Original reference price for depreciation context
`calculate_value_range(used_car_id, comparable_ids)`	Deterministic Django service; writes result to `state["valuation"]`	Range, median, sample size, data-quality flag

Output contract: `{status, low, median, high, sample_size, price_basis: "asking", comparable_post_ids, msrp_reference, limitations}`. Below a minimum sample (e.g. 3 comparables) return `status: "insufficient_data"` with whatever MSRP context exists, never a fabricated range.

Constraints: the LLM never invents prices or does the arithmetic; the range comes from the comparables' distribution, not a fixed ±10% band. Index `post_data` keys used for filtering (GIN index or extracted columns) so the search stays in SQL.

Agent 3 — Buyer Recommendation

Agent 3 saves the buyer's needs as a `buy_issues` record, then matches them against published sell posts (used) and the `new_cars` catalog (new), showing only contact fields sellers approved.

Tool	Reads / writes	Purpose
`create_or_update_buy_issue(description, min_budget?, max_budget?, preferences)`	Writes own `buy_issues`	Persist budget and `preferences` JSON (makes, models, `minimum_year`, `maximum_mileage`, `vehicle_type`, `fuel_type`, `location`)
`search_and_rank_used(buy_issue_id, limit=5)`	Reads `posts` (`sell`, `published`)	Hard filters in SQL before `LIMIT` (price ⩽ `max_budget`, year, mileage, make), then soft-preference scoring with a reason per match
`search_new_catalog(buy_issue_id, limit=5)`	Reads `new_cars`	Only when `vehicle_type` is `new` or unset; label results as catalog entries with reference MSRP, not dealer stock
`get_post_contact(post_id)`	Reads `post_data.contact` of a published sell post	Seller-approved phone, email or link only

Workflow: buyer describes needs → Agent 1 confirms budget and must-haves → issue saved → search and rank → Agent 1 presents top matches with reasons → buyer contacts the seller outside the app. Optionally, Agent 5 turns the buy issue into a public buy post so sellers can find the buyer.

Empty state: no matching posts → return `status: "no_results"` plus which constraint eliminated the most candidates, so Agent 1 can suggest relaxing it.

Agent 4 — Seller Recommendation

Agent 4 creates the `sell_issues` record for an owned car, recommends an asking price from Agent 2's stored valuation, and finds buyers only among published buy posts.

Tool	Reads / writes	Purpose
`get_vehicle_details(used_car_id)`	Reads owned `used_cars`	Seller's car
`create_or_update_sell_issue(used_car_id, description, asking_price?, condition_details?, known_damage?, location?, contact_info?)`	Writes own `sell_issues`; server checks `used_cars.owner_id` = session user	Capture disclosures and private contact preferences
`recommend_asking_price(sell_issue_id)`	Reads `state["valuation"]` from Agent 2	Suggested price with evidence; the seller sets the final `asking_price`
`find_and_rank_buyers(sell_issue_id, limit=5)`	Reads `posts` (`buy`, `published`), matching `post_data` budget and preferences	Ranked buyer posts with match reasons
`get_post_contact(post_id)`	Reads `post_data.contact` of a published buy post	Buyer-approved contact only

Workflow: seller names the car → Agent 1 resolves or registers it → sell issue created → Agent 1 runs Agent 2 → Agent 4 recommends a price and lists matching buyer posts → Agent 1 hands off to Agent 5 for the sale-post draft → seller approves.

Dependency: buyers are discoverable only if they published a buy post. Private `buy_issues` are never searched across users, and the agent never invents interested buyers.

Agent 5 — Community Post Generation

Agent 5 turns one issue into a private `posts` draft (`issue_type` + `issue_id`) and publishes it only after the owner approves the exact public snapshot.

Tool	Reads / writes	Purpose
`get_issue_context(issue_type, issue_id)`	Reads the owned issue + its `used_cars` row	Source facts: description, preferences, condition, symptoms, `resolution_details`
`save_post_draft(issue_type, issue_id, title, content, post_data)`	Writes `posts` (`status = draft`); service validates the type/ID pair and ownership	Agent writes the text; backend validates the `post_data` schema per type
`update_post_draft(post_id, title?, content?, post_data?)`	Updates own draft	Apply the owner's edits
`approve_and_publish_post(post_id)`	Sets `approved_at` , then `status = published` , in one transaction	Only after explicit approval in the current turn; idempotent

`post_data` per type

`issue_type`	Snapshot contents
`sell`	Make, model, year, trim, mileage, asking price, approved condition and damage disclosures, location, approved `contact` fields

issue_type	Snapshot contents
buy	Budget range, desired makes/models/year/mileage, location, approved contact fields
repair	Vehicle, symptoms, method, work performed, cost, mechanic name (only if the owner agrees), owner_reported: true

Rules: an AI hypothesis in ai_analysis never appears as the confirmed fix; only resolution_details does. Self-service steps are labelled as owner-reported and unverified. discussion posts are not supported by the current schema.

Agent 6 — Repair & Mechanic Recommendation

Agent 6 opens a repair_issues record on a registered car, learns from published repair posts, and records the owner-reported outcome that later feeds Agent 5.

Tool	Reads / writes	Purpose
create_repair_issue(used_car_id, description)	Writes own repair_issues (status = open)	Start the case
save_diagnosis(repair_issue_id, symptoms, ai_analysis)	Updates own issue	Store extracted symptoms (inferred vs confirmed) and the possibilities with uncertainty
search_similar_repairs(repair_issue_id, limit=5)	Reads posts (repair, published) by make, model, year and symptom terms; pgvector on content optional	Similar community cases, flagged as anecdotes
estimate_repair_cost(repair_type, make, model)	Aggregates post_data.cost from published repair posts	Owner-reported cost range with sample size; insufficient_data when thin
find_nearby_mechanics(location, specialty)	External places API (not in the DB) + mechanics named in published repair posts	Only with a location the user gave

Tool	Reads / writes	Purpose
`record_resolution(repair_issue_id, status, resolution_details)`	Updates `status`, `resolution_details`, `resolved_at`	Mechanic or self-service outcome, as the owner reports it

Workflow: symptoms → targeted questions (cranks, clicks, silent?) → `save_diagnosis` → similar repairs and cost evidence → mechanics if asked → later, the user reports the outcome → `record_resolution` → Agent 1 offers a repair post via Agent 5.

Safety: brake failure, smoke, fuel leaks and overheating get immediate stop-driving guidance, not DIY steps. No guaranteed diagnosis or cost.

Tool → model → endpoint mapping

Every write tool maps to an existing planned DRF endpoint, so the chat and the web UI share one service layer; read-only search tools need three new internal service functions.

Agent tool	Model	DRF endpoint / service
`register_used_car`	`used_cars`	`POST /api/cars/used/`
`get_vehicle_details`	`used_cars`	`GET /api/cars/used/{id}/` (owner-scoped)
`get_catalog_reference`, `search_new_catalog`	`new_cars`	`GET /api/cars/new/`
`create_or_update_buy_issue`	`buy_issues`	`POST` / `PATCH /api/issues/buy/`
`create_or_update_sell_issue`	`sell_issues`	`POST` / `PATCH /api/issues/sell/`
`create_repair_issue`, `save_diagnosis`	`repair_issues`	`POST` / `PATCH /api/issues/repair/`
`record_resolution`	`repair_issues`	`PATCH /api/issues/repair/{id}/resolution/`
`save_post_draft`	`posts`	`POST /api/posts/generate/`
`update_post_draft`	`posts`	`PATCH /api/posts/{id}/`
`approve_and_publish_post`	`posts`	`POST /api/posts/{id}/approve/`

Agent tool	Model	DRF endpoint / service
`search_comparable_posts`, `search_and_rank_used`, `find_and_rank_buyers`, `search_similar_repairs`, `estimate_repair_cost`	`posts` (published only)	New service: `post_search_service` (filters `post_data` in SQL)
`get_post_contact`	`posts.post_data.contact`	`GET /api/posts/{id}/` (public fields only)
`calculate_value_range`	—	New service: `valuation_service`
`find_nearby_mechanics`	—	New service: `places_client` (external provider)

Implementation rules

1. **Agents select tools; Django executes.** Filtering, ranking, valuation and every mutation live in the service layer.
2. **Identity comes from the session.** Tools read `state["user_id"]`; owner and creator IDs are set server-side, matching the DRF serializer rule.
3. **Cross-user reads touch published posts only.** Agents never query another user's `used_cars`, issues or drafts.
4. **Contact data leaves only through `post_data.contact`.** `sell_issues.contact_info` stays private until the owner approves specific fields into a post.
5. **Validate the polymorphic pair.** Every post write checks `issue_type` ∈ {buy, sell, repair}, the row exists in the matching table, and the caller owns it.
6. **Approval is explicit and idempotent.** `approve_and_publish_post` needs a clear yes from the user in the current turn and is safe to retry.
7. **Structured results.** Every tool returns `{status, data, ids, provenance, error}`; empty searches return `no_results`, never invented rows.
8. **Hard filters before `LIMIT`, then soft scoring with a recorded reason.**
9. **Handle deletes.** When an issue is deleted, archive its posts (`status = archived`) in the same transaction to avoid orphaned logical references.
10. **Tool-returned text is untrusted.** Post content cannot override agent instructions or permissions.

Cross-agent journey with database writes

One registered Civic carries the user from repair to sale; each step leaves a row that the next agent builds on.

Step	User says	Agents	Database effect
1	"My 2018 Civic won't start."	Agent 1	Finds or creates `used_cars` row
2	Answers diagnostic questions	Agent 6	`repair_issues` created; `symptoms`, `ai_analysis` saved
3	"The mechanic replaced the battery, $180."	Agent 6	`status = resolved`, `resolution_details`, `resolved_at`
4	"Share my experience."	Agent 5	`posts` draft, `issue_type = repair`
5	"Looks good, publish it."	Agent 5	`approved_at` set, `status = published`
6	"Should I sell it?"	Agent 4, Agent 2	`sell_issues` created; valuation in session state
7	Sets price, approves post	Agent 4, Agent 5	Sell `posts` row published with `post_data.contact`
8	Another user: "Civic under $15k?"	Agent 3	`buy_issues` created; matches the sell post; shows approved contact

Gaps and MVP priorities

The seven-model schema supports all six agents for the MVP; these gaps limit quality, not function.

Gap	Impact	Optional fix later
`buy_issues` has no `contact_info`	Buyer contact must be captured at post approval	Add `contact_info` JSONB to `buy_issues`, mirroring `sell_issues`
No sold-price data	Valuation shows asking prices only	Add `sold_price` + `sold_at` to `sell_issues` when status becomes `sold`

Gap	Impact	Optional fix later
No mechanics table	Mechanic results depend on an external API	Add a `mechanics` model if curated listings are needed
No `discussion` post type	General community chat unsupported	Extend `issue_type`, or allow posts without an issue
`post_data` searched as JSONB	Slower filters at scale	GIN index now; extract `make`, `model`, `year`, `price` columns later
Repair requires a registered car	Quick anonymous troubleshooting blocked	Make `used_car_id` nullable on `repair_issues`

Hackathon MVP order

1. Agent 1 + `register_used_car` + ADK session with server-set `user_id`.
2. Agent 6 → Agent 5 repair flow end to end (create, diagnose, resolve, draft, approve, publish).
3. Agent 4 + Agent 5 sell flow, with Agent 2 returning `insufficient_data` honestly on thin seed data.
4. Agent 3 search against seeded published sell posts and `new_cars`.
5. Seed 20–30 published sell and repair posts so search, valuation and cost tools return real rows.